

Intelligent Job Portal

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Abhay Tripathi – 2520030353

K. Pranav – 2520030477

Team 9 Section 6

Under the Guidance of

Dr. V. Sireesha

Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled “Intelligent Job Portal” is a record of the work of Abhay Tripathi – 2520030353 and K. Pranav – 2520030477, submitted in partial fulfillment of the requirements for the award of B.Tech in Computer Science and Engineering. This report presents the design, implementation and evaluation of the NovaHire intelligent job portal developed for Data Structures and Algorithms 3 during the academic year 2026–27.

Abhay Tripathi – 2520030353

K. Pranav – 2520030477

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project report entitled “Intelligent Job Portal” is the work done and submitted by Abhay Tripathi – 2520030353 and K. Pranav – 2520030477 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science Engineering during the academic year 2026–27.

Signature of the Guide
(Dr. V.Sireesha)

Signature of the HOD

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. V. Sireesha, Professor, Department of Computer Science and Engineering, for guiding the Intelligent Job Portal project and supporting our study of Data Structures and Algorithms 3.

We thank Dr. P. Venkateswara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, and the department for providing an academic environment in which we could connect algorithm design with software implementation.

We also acknowledge the authors of the Java and browser platform documentation consulted for the implementation. The project helped us practise modular programming, algorithm analysis, interface design and systematic verification.

Abhay Tripathi – 2520030353

K. Pranav – 2520030477

Abstract

Intelligent Job Portal, presented as NovaHire V4, is a local recruitment demonstration that connects a browser interface to a Java implementation of advanced data structures and algorithms. It supports job discovery, profile-based recommendations, explanations of matched and missing skills, and planning under limited application time. The project applies the concepts of Data Structures and Algorithms 3 to a practical search and allocation problem.

The application uses the JDK built-in HTTP server to serve HTML, CSS and JavaScript and to expose JSON endpoints. Thirty seed job records are loaded from a CSV file into a custom dynamic array. Exact text search uses KMP, the Z algorithm or Rabin–Karp; multi-skill coverage uses Aho–Corasick with explicit token boundaries. Edit-distance similarity contributes to title and location matching, while randomized QuickSort orders recommendation scores. Bounded subset-state dynamic programming supports skill-upgrade planning, and a knapsack formulation selects applications within an hour budget. Additional endpoints demonstrate suffix-array indexing, Edmonds–Karp maximum flow, reservoir sampling and parallel reduction.

The interface provides candidate, recruiter and administrator views. Candidate profiles, saved jobs and application state persist in browser localStorage. Recruiter-created jobs are searchable immediately but remain in memory for the current Java server session. Recruiter candidate cards, interview schedules, employer verification and report-review panels include local demonstration data. The system is therefore evaluated as an academic prototype, with operational search and planning services rather than a production hiring platform.

Fresh verification completed all 32 Java self-tests and 17 HTTP/browser checks. The report describes the implemented architecture, module responsibilities, algorithm trade-offs, observed outputs and remaining limitations. Future work includes authenticated roles, durable job storage, real applicant records and evaluation on larger datasets.

Table of Contents

Section	Page
Acknowledgement	4
Abstract	5
Table of Contents	6
List of Figures	7
1 Introduction	8
1.1 Overview	8
1.2 Problem statement	8
1.3 Scope of the project	9
1.4 Objectives	9
2 System Requirements	10
2.1 Functional Requirements	10
2.2 Non-Functional Requirements	11
3 System Design	12
3.1 High-level Architecture Diagram	12
3.2 Description of each layer	13
4 Implementation	15
4.1 Module-wise implementation	15
4.2 Front-end logic UI rendering and state management	16
4.3 API integration	17
4.4 Algorithms and data structures	18
4.5 Execution and project structure	21
5 Results	22
6 Challenges & Limitations	27
6.1 Issues faced during development	27
6.2 Solutions applied	27
6.3 Limitations of the System	28
7 Conclusion & Future Work	29
7.1 Conclusion	29
7.2 Future Work	29
7.3 Skills learned during development	29
References	30

List of Figures

Figure	Page
Figure 1.1 Recommendation pipeline	8
Figure 3.1 Implemented system architecture	12
Figure 3.2 Runtime object model	14
Figure 5.1 Candidate overview	23
Figure 5.2 Job discovery and filters	23
Figure 5.3 Job detail and match explanation	24
Figure 5.4 Recruiter dashboard	25
Figure 5.5 Anonymous candidate presentation	25
Figure 5.6 Administrator dashboard	26

1 Introduction

1.1 Overview

NovaHire V4 implements an intelligent job portal as a local Java web application. A job seeker enters skills, location and a desired title; the system calculates a relevance score for each job and presents ranked opportunities. Recruiter and administrator views demonstrate the surrounding hiring workflow, while Java services perform the search and optimization tasks. The project is maintained in the Team 9 Intelligent Job Portal repository.

The operational architecture is browser presentation, Java HTTP handling, Java service and algorithm processing, and an in-memory repository initialized from CSV. The web edition does not require Spring Boot, React, PostgreSQL, MongoDB, Docker or an external API. The console edition remains available as a second way to exercise the same algorithmic services. Figure 1.1 summarizes the recommendation pipeline used by the running implementation.

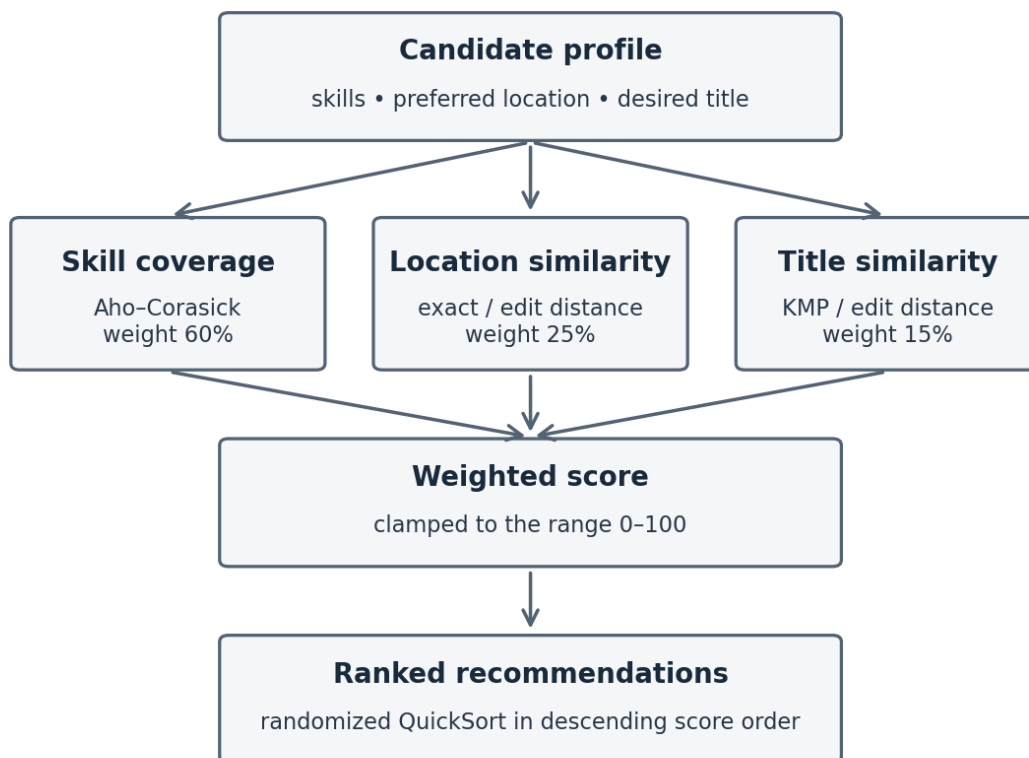


Figure 1.1 Recommendation pipeline

1.2 Problem statement

A candidate may have to scan many vacancies to determine which requirements match their profile. Exact keyword search alone cannot explain suitability, handle spelling mistakes or decide how to spend limited application time. Recruiters also need a way to demonstrate candidate allocation under vacancy constraints. The project addresses these problems through explicit scoring, string-search techniques, dynamic programming and graph optimization, with results presented through a usable interface.

1.3 Scope of the project

The project scope is a reproducible academic recruitment prototype. Seed job descriptions, skills, salaries, experience requirements and vacancies come from `data/jobs.csv`. Search and recommendation process these records and any jobs published during the current server session. The application stores candidate-facing state in the same browser that runs the interface; it does not synchronize profiles or applications across devices.

The backend offers exact and fuzzy search, recommendations, a skill planner, sampling, allocation and optimization services. The UI adds filters, saved jobs, comparison, a local application pipeline and career insights. Recruiter and administrator views demonstrate dashboard presentation and selected interactions. Their sample metrics and candidate identities are not evidence of actual hiring activity or independently verified employers.

1.4 Objectives

Objective	Implementation
Search job text efficiently	Compare KMP, Z and Rabin–Karp over the corpus.
Explain profile suitability	Show weighted skill, location and title scores with skill gaps.
Prioritize limited effort	Use 0/1-knapsack planning under an application-hour budget.
Recommend skill upgrades	Evaluate bounded subsets of missing skills.
Demonstrate vacancy allocation	Build a capacity graph and run Edmonds–Karp.
Apply DSA without collection imports	Use custom Java arrays, queues and supporting structures.
Provide a reproducible demonstration	Launch locally, run self-tests and expose verifiable API outputs.

Success is assessed by correct outputs, agreement between equivalent search methods, feasibility of plans and assignments, and working communication between browser and server. It is not assessed by claimed employment outcomes or by unsupported production-scale performance figures.

2 System Requirements

2.1 Functional Requirements

ID	Requirement	Scope
FR1	Load and list jobs	Read seed CSV at startup and return job JSON.
FR2	Search and filter	Backend text search; browser location, salary, experience and work-mode filters.
FR3	Rank opportunities	Score every job for a supplied profile and sort descending.
FR4	Explain a match	Return score components and matched/missing skill arrays.
FR5	Maintain candidate state	Save profile, saved IDs and applications in localStorage.
FR6	Compare opportunities	Compare up to three selected jobs in a browser drawer.
FR7	Plan learning and applications	Run subset-state skill planning and hour-budget optimization.
FR8	Publish a vacancy	Validate form parameters and insert into the runtime repository.
FR9	Demonstrate advanced algorithms	Expose benchmark, suffix, sampling, allocation and parallel endpoints.
FR10	Demonstrate other roles	Show recruiter and admin panels with local sample data.

Candidate inputs include skill text, a preferred location and a desired title. Published job inputs include title, company, location, skills, description, salary, experience, vacancy count, application hours and work mode. A successful POST /api/jobs returns the created job with HTTP 200. The frontend then reloads jobs and recalculates recommendations.

The application role switch changes presentation; it does not create an authenticated session or enforce authorization. Saved jobs and submitted applications are candidate-side interactions rather than backend recruitment transactions. These boundaries are part of the functional scope.

2.2 Non-Functional Requirements

Requirement	Design response and limit
Correctness	Self-tests cover algorithms, structures and integration; fresh HTTP checks cover service outputs.
Usability	Job cards, drawers, filters and role-specific views use product terminology.
Portability	JDK 17 or later, a modern browser and project files are sufficient.
Maintainability	Java packages separate models, structures, algorithms, services, console and HTTP handling.
Reproducibility	Seed CSV remains unchanged by runtime publications.
Responsiveness	CSS provides desktop, tablet and mobile rules; local server binds to loopback.
Data safety	Canonical file checks restrict static asset reads to the web root; input parameters have bounds.
Performance	Algorithms expose operation counts and timings; the small corpus does not validate large-scale throughput.
Persistence	Browser state survives refresh; runtime job additions do not survive a server restart.

The development environment requires JDK 17 or later with both java and javac, plus VS Code or another editor. The launcher checks these commands, compiles source files and runs the self-tests before starting the web server. It uses port 8080 by default and tries ports through 8089 when the default port is occupied. A normal browser accesses the address printed in the startup log.

No specific minimum RAM or CPU benchmark is established in the repository. The report therefore does not assign an unmeasured hardware requirement or response-time guarantee. Public deployment would require security, durable storage and concurrency controls beyond this local demonstration.

3 System Design

3.1 High-level Architecture Diagram

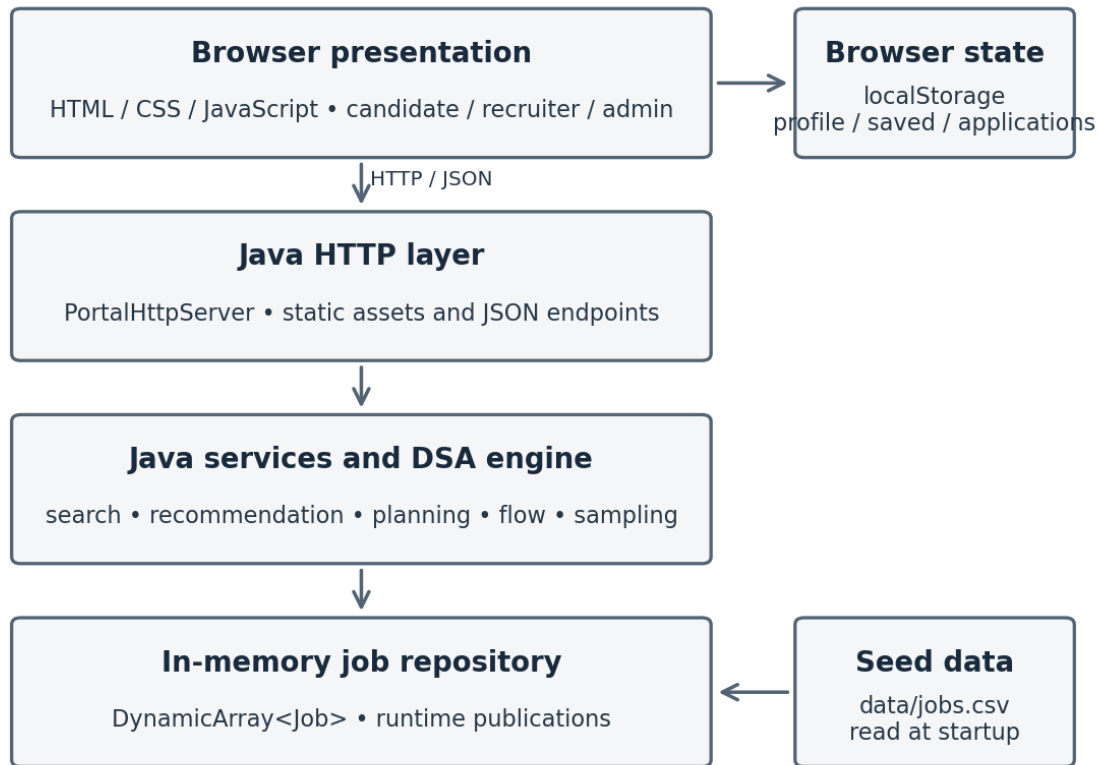


Figure 3.1 Implemented system architecture

The browser loads its static assets and calls JSON endpoints on the same Java server. PortalHttpServer routes requests to the search engine, recommendation engine and independent algorithm modules. JobRepository supplies the shared DynamicArray of jobs. Candidate state follows a separate browser-storage path; it is not written to CSV by the backend.

The server binds to 127.0.0.1. WebMain locates the CSV and web directories, loads the repository, creates the server and prints the selected URL. A server restart rebuilds the repository from the seed data. This lifecycle keeps classroom demonstrations repeatable but means recruiter publications are session-only.

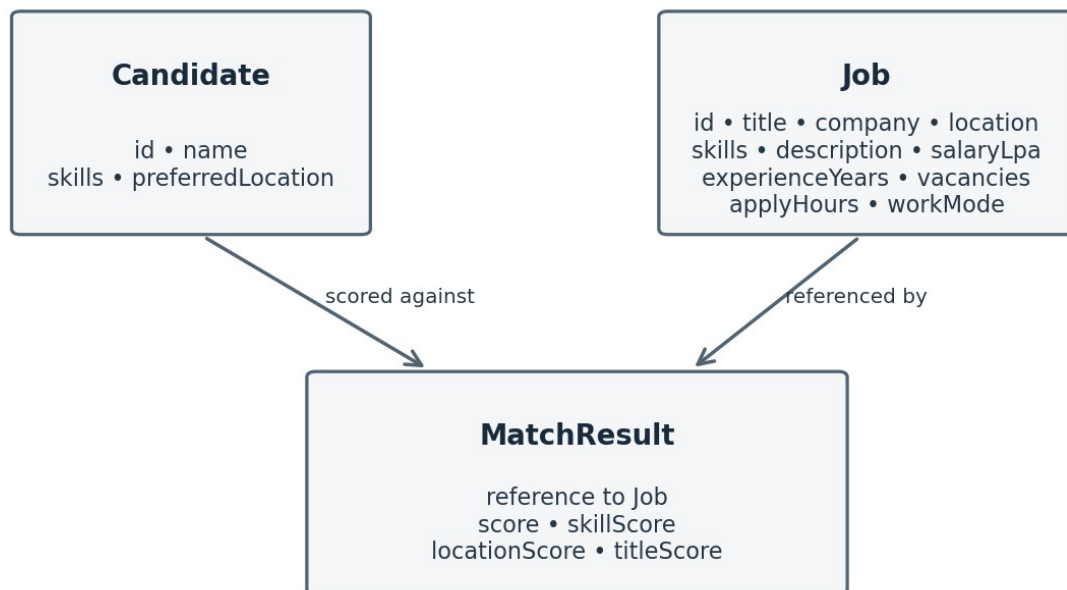
3.2 Description of each layer

Layer	Files	Responsibility
Presentation	web/index.html; assets/styles.css	Defines views, navigation, forms, cards, drawers and responsive layout.
Browser logic	web/assets/app.js	Manages state, events, Fetch calls, filters and localStorage.
HTTP	web/WebMain.java; PortalHttpServer.java	Starts the local server, serves assets, parses parameters and returns JSON.
Services	JobRepository; JobSearchEngine; JobRecommendationEngine; CandidateJobMatcher	Loads jobs and coordinates search, scoring and assignment.
Algorithms	algorithms/*	Implements text processing, DP, flow, randomized methods and reduction.
Structures and models	structures/*; model/*	Stores jobs, traversal queues, keys and typed domain objects.
Data source	data/jobs.csv	Provides 30 reproducible seed job records.

The Java HTTP layer uses JDK `HttpServer` and `HttpHandler` rather than a framework controller. Each API context is registered in `registerRoutes()`. Its wrapper sends HTTP 400 for `BadRequest`, HTTP 500 for unexpected exceptions, and HTTP 200 for successful responses. `OPTIONS` requests return an empty 204 response. Unknown API paths are handled by the static handler with a JSON 404 response [1].

The browser `api()` helper constructs query strings, calls `fetch()`, parses JSON and throws an error when HTTP status or the response `ok` flag indicates failure. Frontend render functions update specific containers instead of navigating to separate server-rendered pages. Role navigation hides and shows the corresponding views [2].

3.3 Data and object design



Runtime Java models; this implementation has no relational database tables.

Figure 3.2 Runtime object model

Job is the central record. Its immutable fields describe identity, role, company, location, skills, description, salary in LPA, required experience, vacancies, application hours and optional work mode. Candidate contains identity, name, skills and preferred location. MatchResult references a Job and stores the total and component scores. These are Java objects, not relational tables or ORM entities.

The CSV header supplies ten fields; work mode is absent in seed rows and is derived in the browser when unspecified. Published jobs can carry a validated work mode directly. Skill strings use a vertical-bar separator internally, with trimming and case normalization for matching. The CSV loader uses simple comma splitting rather than a general quoted-CSV parser.

The browser state object keeps jobs and lookup maps, role, profile, saved IDs, selected comparison IDs and application objects. Profile, saved IDs, role and application state use the novahire.v4 localStorage namespace. Comparison selection is transient. Storage is tied to the browser origin and can be cleared by the user [3].

4 Implementation

4.1 Module-wise implementation

Module	Main implementation
Job repository	JobRepository.load() reads CSV, add() appends a runtime record, nextId() scans IDs and findById() scans jobs.
Search	JobSearchEngine.exactSearch() chooses KMP, Z or Rabin–Karp and checks each job searchableText().
Recommendation	JobRecommendationEngine.score() computes components; recommend() sorts the complete result array.
Assignment	CandidateJobMatcher.assign() creates capacities and extracts assignments from maximum flow.
Skill planning	SkillUpgradePlanner.plan() evaluates bounded missing-skill subsets.
Application planning	KnapsackOptimizer.exact() and halfApproximation() compare exact DP with a density-based heuristic.
Console	Main and ConsoleUI expose the academic algorithms through a menu.
Web server	WebMain and PortalHttpServer connect services to the browser.
Verification	SelfTest checks algorithms, custom structures and project integration.

Package separation allows the same data and algorithm classes to serve both the console and browser editions. Models contain record properties; algorithm classes perform specific computations; service classes coordinate those computations. The HTTP layer also contains compact helpers for parsing form/query parameters and serializing JSON without external dependencies.

The implementation does not import java.util collections in its Java source. Its DynamicArray is used throughout the repository and service results. IntQueue is used for breadth-first processing in Aho–Corasick failure construction and Edmonds–Karp. CustomHashTable is implemented and tested, but the active web search engine still scans job records; the table is not a deployed inverted job index.

4.2 Front-end logic UI rendering and state management

The frontend uses plain JavaScript with a centralized state object and DOM helpers. `init()` binds events, restores the stored role/profile and loads core job data. `loadCore()` obtains jobs, insights and recommendations. Render functions build job cards, saved views, application rows, company summaries and role dashboards. `escapeHtml()` protects dynamic text inserted into HTML templates.

The `api()` helper wraps the Fetch API. It URL-encodes non-empty parameters, parses the JSON response and rejects unsuccessful responses. Candidate edits trigger `saveState()`, `refreshMatches()` and relevant render functions. Browser Map and Set objects are used for lookups and selections; the restriction on built-in collections applies to Java source, not to browser JavaScript.

A delegated document click handler processes navigation, opening job details, saved-job toggles, comparison selection and local application submission. It also handles role selection and demonstration shortlist/admin actions. Separate listeners handle search forms, profile saving, modal submission, anonymous mode and filters. Escape closes drawers, notifications and the job-posting modal. Ctrl/Cmd+K focuses global search.

Saved IDs and application state are serialized into `localStorage`. Applying creates a local object containing job ID, status and date; it is not sent to an application-management API. The application screen consequently demonstrates a pipeline in one browser. It does not represent independently synchronized employer decisions.

Job enrichment derives sample applicant counts, posted-day values, competition levels and some classifications from record attributes. All seed companies are marked verified by this display helper. Recruiter applicants and interview schedules are hard-coded examples. These enrichments make the UI demonstrable but are not live labor-market data.

4.3 API integration

Endpoint	Inputs and output
GET /api/health	Application version, job count and engine status.
GET /api/insights	Corpus counts, vacancy total and salary summary.
GET /api/jobs	Optional limit; returns job records.
POST /api/jobs	URL-encoded job form; validates and inserts a runtime job.
GET /api/search	q and algo; exact matches, timing and operation family.
GET /api/fuzzy	q and distance; typo-tolerant title matches.
GET /api/recommend	skills, location, title, name and limit; ranked score components.
GET /api/skill-plan	skills, location, title and budget; selected missing skills and projected utility.
GET /api/benchmark	q; match counts, operations and timings for three search methods.
GET /api/suffix	q; substring result, suffix count and index timings.
GET /api/featured	k; a reservoir sample of jobs.
GET or POST /api/allocate	Candidate fields and threshold; capacity-constrained assignments.
GET or POST /api/optimize	Profile and hour budget; exact and heuristic plans.
GET /api/parallel	threads; sequential and parallel salary totals.

POST payloads use application/x-www-form-urlencoded, as produced by URLSearchParams. The parameter helper merges POST form fields with query parameters and limits body reads to 64 KiB. Blank required text produces a validation error; numeric parameters are parsed with defaults and bounds. The report distinguishes these actual form contracts from a JSON request-body API.

4.4 Algorithms and data structures

4.4.1 Exact and fuzzy text search

KMP constructs a longest-prefix-suffix table so mismatch handling can reuse previous comparisons. The Z algorithm compares the query against text positions through a combined-string Z array. Rabin–Karp maintains rolling hashes and verifies character matches on hash hits. All three are available through `exactSearch()` and are compared by `benchmark()` using the same searchable job corpus.

For J jobs, exact search visits each record. KMP and Z take $O(T + Jm)$ time across total searched text length T and query length m , including repeated pattern work per record. Rabin–Karp has expected linear-style behavior but can incur extra verification work from collisions. These routines do not provide $O(1)$ job lookup. Operation counts and agreement on returned matches are useful comparisons; timings vary with JVM warm-up.

Fuzzy title search splits query and title into words. Each query word is compared to every title word using Wagner–Fischer edit distance, and its minimum distance contributes to the query score. A job is accepted when the accumulated distance is within the requested tolerance. This is word-wise approximate title matching, not a learned semantic search model.

The suffix index concatenates searchable job strings into a corpus. Prefix doubling with merge sorting builds suffix order in $O(n \log^2 n)$; LCP construction records shared prefixes. A query uses binary search and up to m character comparisons per step, giving $O(m \log n)$. The current suffix endpoint rebuilds its index for each request, so reported query timing alone does not represent total request cost.

4.4.2 Multi-skill matching

Aho–Corasick builds a trie, failure links and output information for candidate skills. Skills are normalized into patterns such as `|java|` so Java does not falsely match JavaScript. Distinct matched patterns are counted against the required job-skill list. The denominator is the number of required skills; adding unrelated candidate skills does not reduce coverage. An `IntQueue` supports failure-link construction.

4.4.3 Recommendation scoring and ranking

The recommendation engine scores every job using three components. Required-skill coverage is the percentage of job skills found among the candidate skills. Location receives 100 for an exact case-insensitive match, 50 when no preference is supplied, and edit-distance similarity otherwise. Title receives 100 when KMP finds the desired title as a substring; a blank desired title receives 50, and other titles use edit-distance similarity.

```
Match score = 0.60 × skill score
              + 0.25 × location score
              + 0.15 × title score
```

Each output score is clamped to 0-100.

This is an explicit rule-based scoring model. The weights are implementation choices rather than weights learned from hiring outcomes. Salary, experience, applicant count and employer verification do not enter this backend formula. Some of those fields support separate browser filters or opportunity lenses.

RandomizedQuickSort chooses a pivot with the project LCG random generator, partitions MatchResult entries in descending score order and recurses on the smaller side while iterating over the other. Expected sorting time is $O(J \log J)$, with $O(J^2)$ worst-case comparisons. It ranks the full result array; it does not use a heap-based top-K procedure. Equal scores are not given a stable ordering guarantee.

4.4.4 Skill upgrade planning

SkillUpgradePlanner collects missing skills from up to eight ranked jobs, limits the universe to 15 skills and caps the learning budget at four. Each bitmask represents a chosen subset. Utility increases with added required-skill coverage and includes a completion bonus of 35 when a job becomes fully covered. The recurrence removes one selected bit, reuses the smaller subset utility and adds that skill's marginal benefit.

The bounded state space has at most 2^{15} masks. Masks exceeding the skill budget are skipped. The endpoint constructs an upgraded profile and recalculates recommendations so the user can compare before/after scores. The projected utility is an internal objective, not an employment probability or a guarantee that learning the chosen skills will obtain an offer.

4.4.5 Network flow and application optimization

CandidateJobMatcher builds a directed capacity graph with source, candidate nodes, job nodes and sink. Source-to-candidate capacity is one. Candidate-to-job edges exist when the rule-based match score meets a threshold. Job-to-sink capacity is the job vacancy count, with a minimum of one in this implementation. Edmonds–Karp uses breadth-first augmenting paths and residual capacities until no additional flow is possible.

The result maximizes the number of feasible assignments under these capacities, not the total quality of assigned matches. An assigned job index is extracted from positive candidate-to-job flow. The implementation stores capacities and flows in matrices. Matrix neighbor scanning adds practical cost compared with adjacency-list traversal, so the usual $O(VE^2)$ Edmonds–Karp analysis should be interpreted alongside this representation.

For application strategy, a job’s rounded match score is its value and applyHours is its weight. The exact 0/1-knapsack DP compares skipping and taking a job, then traces the table backward to recover selected items. Its time and memory are $O(JH)$ for J considered jobs and hour budget H . The web endpoint considers only the top 12 recommended jobs and bounds H to 40.

A second method scans value-per-hour densities and compares the resulting feasible selection with the best feasible single job. The code names this halfApproximation and labels it a 1/2 approximation. The report treats the observed comparison as implementation evidence; checking the bound on demonstration inputs does not prove a universal approximation guarantee for arbitrary data.

4.4.6 Randomized sampling and parallel reduction

Reservoir sampling initializes a sample of k jobs, then considers each remaining item once for possible replacement. It requires $O(J)$ time and $O(k)$ sample space. Its random values come from a custom LCG, which is suitable for a classroom demonstration and is not a cryptographic or independently validated randomness source. ParallelScoreReducer divides salary values into chunks, starts worker threads, joins them and sums their partial results. Thread startup overhead can exceed the benefit on 30 records.

4.5 Execution and project structure

Open the innermost IntelligentJobPortal-DSA3-v4 directory in VS Code. This directory contains src, web, data, docs and the launcher scripts. Do not run the launcher from a parent containing only another nested folder. A JDK installation must expose java and javac to the terminal. The normal project launcher compiles all source files, runs SelfTest and starts WebMain.

```
java -version
javac -version
.\run-web.bat
```

Default browser address: <http://localhost:8080>

Optional explicit port: `.\run-web.bat 8085`

Console edition: `.\run.bat`

The Windows launcher adds the `jdk.httpserver` module when compiling and running. It writes class files to `out`, builds a source-file list and stops on compilation or self-test failure. WebMain opens a browser when a graphical desktop is available; in a headless environment the URL can be opened manually. Ctrl+C stops the server. It leaves `seed.jobs.csv` unchanged.

Directory	Contents
src/com/dsa/jobportal/model	Candidate, Job and MatchResult
src/com/dsa/jobportal/structures	DynamicArray, IntQueue and CustomHashTable
src/com/dsa/jobportal/algorithms	String, DP, flow, optimization, randomized and parallel classes
src/com/dsa/jobportal/service	Repository and coordinating engines
src/com/dsa/jobportal/web	WebMain and PortalHttpServer
web	index.html and JavaScript/CSS assets
data	jobs.csv seed corpus
docs	Repository project overview and verification notes

In this report's fresh execution, source was compiled with Java 17 and the self-tests were run from the project root so the relative CSV path resolved. Browser/API verification used temporary port 8097. This verification port is not a change to the repository's default launcher settings.

5 Results

5.1 Verification results

Fresh verification on 5 October 2026 compiled the repository's Java sources and passed all 32 SelfTest checks. Seventeen additional HTTP/browser checks passed against the local Java server. The verification included the seed corpus, search and recommendation, planning and allocation endpoints, validation responses, browser role views and runtime job publication. Source snapshot: commit 60c92481407da657a9d8003eea09c87c1d38b4a2.

Check group	Observed result
Java compilation	Successful on Java 17.
Algorithm and integration self-tests	32/32 passed.
Seed data	30 jobs, 22 companies, 5 locations and 78 vacancies.
API success paths	Health, jobs, insights, search, fuzzy, recommendation, skill-plan, benchmark, suffix, featured, allocate, optimize and parallel returned HTTP 200.
Validation	Missing exact-search query returned HTTP 400.
Unknown API route	Returned JSON HTTP 404.
Publication	POST inserted a Remote job and exact search found it immediately.
Browser integration	Candidate, recruiter and admin views opened with no captured JavaScript page errors.

These results validate the exercised paths on the local demonstration dataset. They do not establish production readiness, multi-user isolation, exhaustive input coverage or a measured large-scale speedup. The original CSV was kept unchanged. Temporary publication data was discarded when the verification server stopped.

5.2 Candidate interface

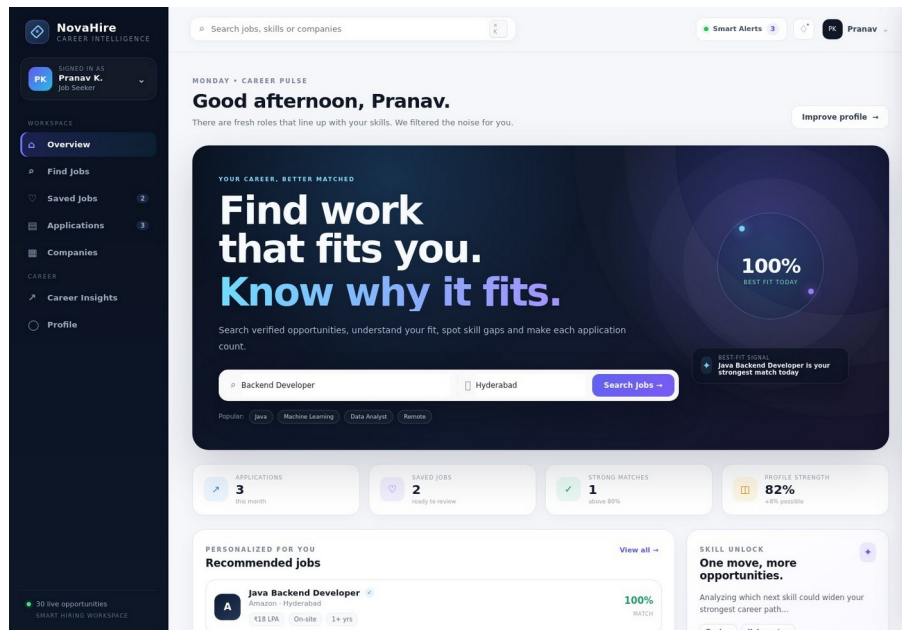


Figure 5.1 Candidate overview

The overview exposes career discovery, featured opportunities and candidate navigation. Job cards show company, role, location and a match indicator derived from the loaded profile. The screenshot is a fresh capture of the repository’s running UI, using its existing sample profile and dataset.

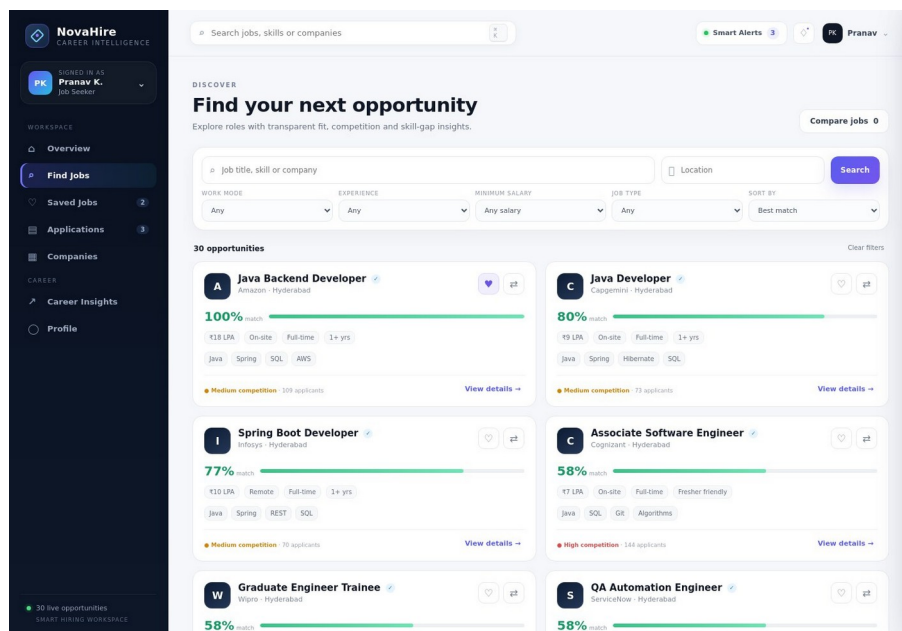


Figure 5.2 Job discovery and filters

The job-discovery view combines backend search with browser filters and sorting. Search services inspect actual job text; competition and applicant indicators remain derived demonstration values. The interface is therefore useful for showing interactions without presenting these values as live employer statistics.

5.3 Match explanation and skill planning

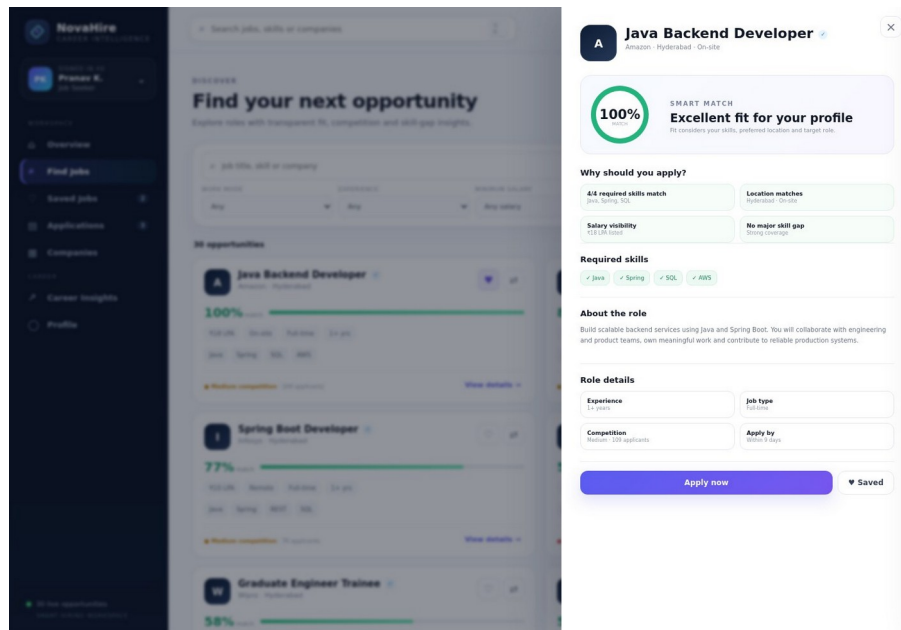


Figure 5.3 Job detail and match explanation

The job drawer displays the vacancy and explains suitability with matched and missing skills. Backend recommendation responses include total, skill, location and title scores, supporting an explanation beyond a single percentage. A score of 100 indicates full agreement under the rule-based formula, not guaranteed selection by an employer.

For the fresh test profile Java, Spring, SQL and AWS, preferred location Hyderabad and desired title Backend Developer, Java Backend Developer was returned with skill, location and title scores of 100; the weighted score was also 100. Its description contains Spring Boot as a job requirement, but that does not make Spring Boot the technology used by this portal's backend.

A separate skill-planning request used Java and SQL, Hyderabad and a budget of two skills. It selected Git and Algorithms from an 11-skill universe, evaluated 67 budget-feasible states, and increased internal utility from 350 to 510. The top recommendation score changed from 62.5 to 92.5 after constructing the upgraded profile. These are reproducible outputs for those inputs, rather than labor-market predictions.

5.4 Recruiter and administrator views

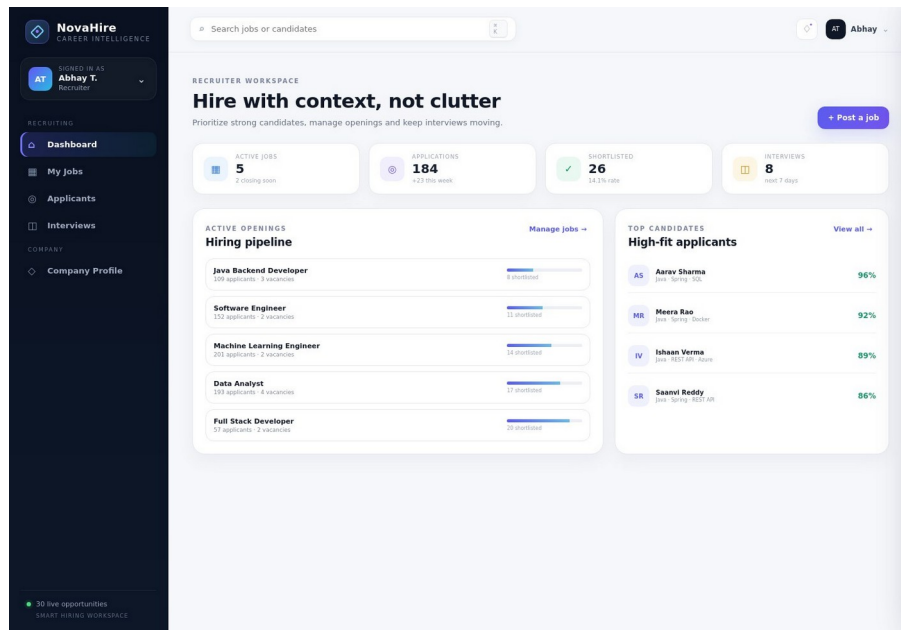


Figure 5.4 Recruiter dashboard

The recruiter dashboard combines loaded vacancies with sample candidate metrics. Job publishing is connected to a real backend insertion path. In contrast, applicant cards, shortlist counts and interview schedules are local demonstration content; several View, Edit and Close controls do not perform a durable backend modification.

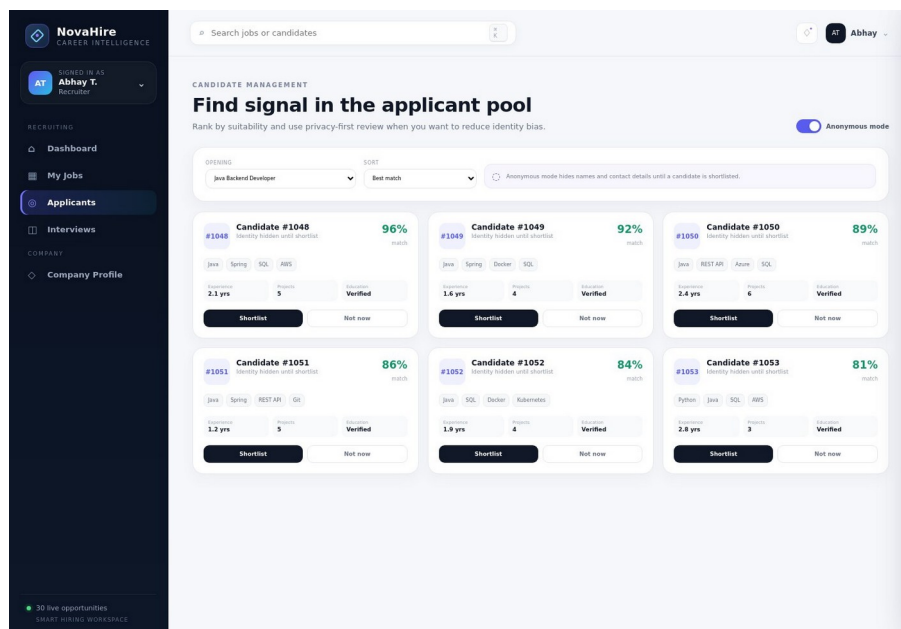


Figure 5.5 Anonymous candidate presentation

Anonymous mode hides names in the presented candidate cards. It is a UI transformation over sample applicant objects, not anonymization of a server-held applicant database. The shortlist action displays a notification; it does not persist a real hiring decision.

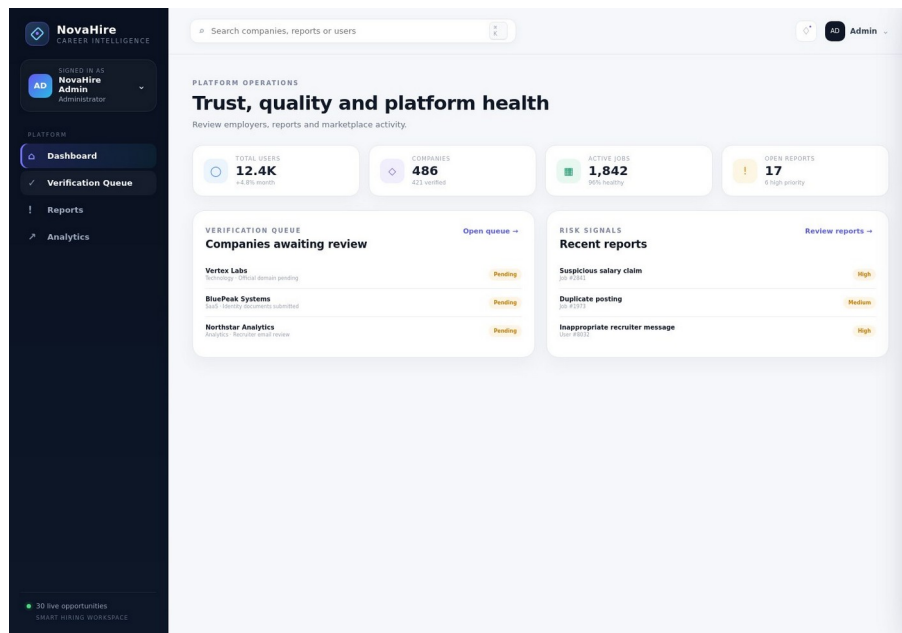


Figure 5.6 Administrator dashboard

The administrator dashboard presents employer-review and report-review demonstrations. Its queue entries and reports are sample arrays in app.js. Review controls display local feedback, with no verified-document storage or audit trail. The report records the screen as an implemented presentation feature while keeping operational administration as future work.

5.5 Optimization and aggregation outputs

Experiment	Observed output
Application plan	With Java/Spring/SQL/AWS and an eight-hour budget, the exact plan used eight hours and achieved total value 245.
Heuristic comparison	The comparison plan achieved value 245 on the same input; observed ratio to exact value was 1.0.
Candidate allocation	Two supplied candidates were assigned under a score threshold of 30 and available capacities.
Parallel reduction	Sequential and four-worker salary totals both equaled 443.
Corpus salary summary	Mean listed salary was 14.767 LPA; highest listed salary was 28 LPA.

The salary totals and means summarize seed job listings, not salaries earned by candidates or independently collected industry statistics. Matching totals establish functional consistency of reduction, not a speedup claim. Similarly, one heuristic ratio of 1.0 demonstrates this input's outcome and does not imply the heuristic is always optimal.

6 Challenges & Limitations

6.1 Issues faced during development

The central design challenge is connecting a broad DSA syllabus to a focused recruitment workflow. Exact matching, approximate similarity, planning, flow and sampling solve different problems; combining them under a single undefined “intelligence” claim would obscure the actual implementation. The project instead exposes discrete Java modules and keeps the main interface focused on candidate decisions.

Text representation creates another challenge. A simple substring match can treat Java as part of JavaScript, while extra candidate skills can incorrectly reduce precision-based scores. Profile input uses comma-separated text, but internal skills use vertical bars. A consistent normalization path and explicit skill boundaries are therefore necessary.

The local implementation must also remain reproducible. Recruiter publication needs to update search immediately without altering the seed corpus used by self-tests. Browser application state needs to survive refresh without requiring a database installation. These choices lead to split persistence, with different lifetimes for seed records, runtime jobs and browser state.

6.2 Solutions applied

Challenge	Implemented response
False skill matches	Boundary-delimited Aho–Corasick patterns separate Java from JavaScript.
Extra skills reducing coverage	Required job skills form the score denominator.
Typo-tolerant retrieval	Word-wise edit-distance title search supplements exact search.
Exponential skill subsets	Missing-skill universe capped at 15 and budget capped at four.
Oversized application planning	Only top 12 recommendations and at most 40 budget hours are considered.
Local port conflict	Default startup tries ports 8080–8089.
Modal interference	Explicit hidden-state CSS and close/Escape/backdrop handlers.
Reproducible seed data	Published jobs append to the shared runtime array without rewriting CSV.

6.3 Limitations of the System

The portal is not an authenticated multi-user service. Role switching is client-side presentation; API publishing has no recruiter authorization. Candidate application state stays in one browser, and published vacancies disappear on restart. Sample applications, employer badges, notifications, candidate scores and admin queues must not be presented as independently verified external data.

Search scans every job. The custom hash table is tested but not used as a live inverted index. The suffix endpoint rebuilds its index for each request. Recommendation sorts all scores instead of selecting top-K with a heap. These facts limit claims of constant-time search or unusually fast large-corpus recommendations.

Score weights and planner utility are fixed rules. Experience and salary do not contribute to the recommendation formula, and edit-distance similarity does not capture semantic equivalence. Skill-planner truncation can omit a useful skill beyond the bounded universe. Max flow optimizes feasible assignment count, not total match quality. The knapsack comparison requires broader analysis before making a universal approximation-bound claim.

Data handling is intentionally lightweight. The loader does not implement quoted CSV, persistent job updates, a transactional application store or distributed concurrency. The server provides basic parameter bounds and error messages rather than complete production-grade security and abuse protection. It binds to the local loopback address and is not deployed for remote public access.

The seed corpus contains 30 jobs. The report does not claim statistical significance, measured recruitment improvement, algorithmic superiority on all inputs, verified market salaries or exhaustive accessibility/mobile-device validation. Larger datasets, representative users and failure-mode tests would be needed to establish those outcomes.

7 Conclusion & Future Work

7.1 Conclusion

Intelligent Job Portal demonstrates an end-to-end path from browser interaction to explicit algorithmic computation. NovaHire V4 loads job records, searches text, scores profiles, explains skill gaps and provides bounded planning tools. The Java package structure makes each DSA technique inspectable, while the browser presents its outputs in a recruitment-oriented interface.

Fresh compilation, 32 passing self-tests and 17 HTTP/browser checks support the correctness of the exercised demonstration paths. The achieved outcome is a working local academic prototype with verifiable algorithm behavior. Its present persistence and role boundaries remain clearly identified, so operational features are not confused with production recruitment infrastructure.

7.2 Future Work

The next stage should introduce authenticated candidate, recruiter and administrator identities; server-enforced authorization; durable storage for jobs, profiles, applications and hiring decisions; and explicit privacy controls. Recruiter and admin sample panels can then be connected to actual records with audit trails. Runtime publication should become persistent with validation and edit/delete workflows.

Algorithmic extensions include a reusable text index, measured top-K selection, stable tie handling, larger-corpus evaluation and improved semantic skill normalization. Weighted candidate allocation could optimize match quality as well as assignment count. A formal review of the optimization heuristic and adversarial test cases would clarify which guarantees can be claimed. Resume parsing and machine-learning recommendations should be treated as new work rather than existing functionality.

7.3 Skills learned during development

The project exercises Java object-oriented design, custom structures, pattern matching, dynamic programming, network-flow modeling, randomized algorithms and thread synchronization. It also develops skills in HTTP integration, DOM rendering, browser persistence, input handling, algorithm complexity analysis, test execution and documentation of implementation limits.

References

- [1] Oracle. Java SE 17 documentation: `HttpServer`, `jdk.httpserver` module. <https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html>
- [2] MDN Web Docs. Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [3] MDN Web Docs. `Window.localStorage` property. <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [4] Team 9. Intelligent Job Portal repository, source snapshot 60c92481407da657a9d8003eea09c87c1d38b4a2. https://github.com/abhaytripathi-cj7/KLH_CSE_2026-2027_S6_Team9_Intelligent_job_portal
- [5] Team 9. NovaHire V4 README, docs/PROJECT_OVERVIEW.md and docs/TEST_REPORT.md within the repository project directory.
- [6] Team 9. Java implementation: `service/JobSearchEngine.java`, `JobRecommendationEngine.java`, `CandidateJobMatcher.java` and `JobRepository.java`.
- [7] Team 9. Algorithm source packages: `stringalgorithms`, `dp`, `flow`, `optimization`, `randomized` and `parallel`; `SelfTest.java` and `data/jobs.csv`.
- [8] Department of Computer Science Engineering. Project Documentation format 2026–27, supplied college report template.

External platform documentation was consulted on 5 October 2026. Project-specific descriptions, object fields, scoring weights and limitations are derived from the repository code. The Results chapter includes fresh execution outputs and captured views from that source version.